Spazio riservato alla correzione

1	2	3	4	Totale
/6	/8	/8	/10	/32

utilizzate altri fogli. Utilizzate soltanto lo spazio sottostante. Fogli differenti non saranno in considerazione per la correzione. Non scrivere a matita. Per tutti gli esercizi, descriver plessità asintotica delle funzioni implementate

. Si consideri una coda **Q**, implementata con array **Q[MAX+2], riempita con interi**. S implementi la funzione ricorsiva **void moltiplica2(int Q[])** che, utilizzando una librer di funzioni di accesso alla coda (**da implementare**) prendendo in input Q, restituisce coda con gli elementi nello stesso ordine, sostituendo però ogni coppia di numeri disp consecutivi con il doppio del loro prodotto, dando priorità agli elementi più vicini al fi della coda. Si ricordi che la coda è una struttura dati che permette l'accesso ai suoi dat solo dalla testa.

Esempio: coda iniziale 3|5|7|4|5|5 (3 fine della coda) – coda finale 30|7|4|50|

Si considerino due liste di numeri interi **Lista1** e **Lista2** implementate come list doppiamente puntate e non circolari, utilizzando la seguente struttura

```
struct elemento {
        struct elemento *prev;
        int inf;
        struct elemento *next;}

struct elemento *Lista1,*Lista2;
```

Si implementi una funzione **ricorsiva** che presi in input Lista1 e Lista2

a. rimuova da Lista1 tutti i numeri pari
b. rimuova da Lista2 tutti i numeri dispari.
c. rimuova da Lista1 tutti i numeri multipli di 3 e li inserisca continuamente in testa alla Lista2.
d. Restituisca le due liste modificate.

3. Siano T e P due alberi binari di ricerca, implementati con la seguente struttura a puntatori:

```
struct nodo {
        int inforadice;
        struct nodo *left;
        struct nodo *right;}

struct nodo *T,*P;
```

Verificare che

a. entrambi gli alberi sono degli ABR,
b. entrambi hanno la stessa struttura
c. dato n, la massima differenza tra le chiavi corrispondente tra i due alberi è al più n.

4. Siano **G** e **H** due grafi orientati pesati entrambi con pesi positivi, di **n** vertici 0, 1,..., n-1 e rappresentati con liste di adiacenza utilizzando la seguente struttura:

```
typedef struct graph {
        int   nv;
        edge   **adj; } graph;

        graph *G, *H;
```

```
typedef struct edge  {
        int   key;
        int   peso;
        struct  edge   *next; } edge;
```

a. scrivere in linguaggio C una funzione che, presi in input **G** ne calcoli:
   - il grafo trasposto
   - il grado incidente e adiacente

b. scrivere in linguaggio C una funzione che, presi in input **G e H** restituisca la differenza "esclusiva" di G e H nel seguente modo: per ogni arco in G presente anche in H si sottrae l'attuale peso in G con quello in H e se la differenza è negativa si elimina l'arco da G. Viceversa, se l'arco è in H ma non in G, si inserisce l'arco in H in G. Se invece è in G ma non in H, l'arco rimane invariato.